

Assignment 1:

In libraries or bookstores, there are portals housing records of available books where users can search and filter for a book. This requires labelling books with their metadata. You are a digital transformation specialist trying to use language models to extract relevant metadata, given the book's information in a text file.

Task 1.1

Using your favorite open-source language model framework/library, initialize a local language model on your favorite environment (docker, conda, or just through your IDE). Preferably, we recommend you run your model inference through Python script, instead of CLIs. If the setup is successful, you can do a simple, "Hello, how are you?" and should await a response.

Hint: If you don't have a preferred model, we recommend ***qwen2.5:3b-instruct-q4_K_M***. If you are using Huggingface, **qwen2.5:3b instruct** is enough.

Task 1.2

Using your knowledge or techniques in prompt engineering, try extracting accurate metadata in JSON format from all the test sets provided.

(1.2.1) Then, analyze and comment on the model's response. List as many observations you can make on the model's response.

(1.2.2) Following from **1.2.1**, do you have any comments on improving the response? Did you try anything in the prompt to enhance it? If you do, kindly elaborate on your techniques and how it did fare? Finally, give your overall thoughts on prompt engineering.

The JSON format should have the following keys: **title, author, year_of_publish, publisher, summary, keywords, rating**.

Do take note of each key/object type (like string, float etc.). The JSON extracted from the language model is correct if it can be loaded from string without error using functions like "json.loads" in python.

Additional info:

- Test sets are the .txt files labelled with “test_”
- You are also given two demo samples, labelled with “demo_” as references. Use it however you like if it helps.

Task 1.3:

Using your prompt from Task 1.2, try adjusting the temperature of the language model between 0.0 to 1.0 (both inclusive). Rerun the extraction from Task 1.2 with different temperature values, respectively. *Describe your observations on the model’s response. What inference can you make from this.*

Assignment 2:

You have been appointed to create a clean energy chatbot advisor using Retrieval Augmented Generation (RAG). Complete all instructed tasks to achieve your objectives.

Task 2.1

Setup your favorite open-source vector database for Python scripts. You may populate the database with dummy data to verify if your setup is successful, but make sure the database is empty at the end of this task.

Hint: If you don’t have a preferred vector database, we recommend using Chroma DB with Python for its simplicity. For windows users, if you face a weird bug with Chroma DB, try version **0.5.3**.

Task 2.2

You should have access to a text file called ‘context.txt’. That will be your context information. This text context is too large and has to be split into manageable sizes. Your team is currently using the below function to segregate the context into smaller chunks:

Function 1:
<pre>def fixed_length_chunking(text, chunk_size=100, overlap=20): """</pre>

Splits text into fixed-length chunks by word count, with optional overlap.

Parameters:

text (str): The input text to chunk.

chunk_size (int): Number of words per chunk.

overlap (int): Number of overlapping words between consecutive chunks.

Returns:

List[str]: List of word chunks.

```
"""
```

```
words = text.split()
```

```
chunks = []
```

```
start = 0
```

```
while start < len(words):
```

```
    end = min(start + chunk_size, len(words))
```

```
    chunk = ' '.join(words[start:end])
```

```
    chunks.append(chunk)
```

```
    start += chunk_size - overlap
```

```
return chunks
```

Using the above function with chunk size 20 and overlap 4, split the text into chunks. Then populate those chunks into your vector database. Furthermore, using your language model from Task 1 and the context from your vector database, answer the following questions:

1. "What are the main challenges in renewable energy storage technologies?",
2. "How do governments influence the adoption of renewable energy?",
3. "Why do some communities oppose large-scale wind farms?"

Finally, give your views or comments on the model's response. Do you think the model covers all necessary info from the passage?

Request: Please retrieve only 5 chunks for this task.

Task 2.3

From Task 2.2, do you think the model's response could or should be improved? If you think so, propose and execute your method and compare the model's responses to that in Task 2.2. Also, include your remarks on the results of this comparison.

If you think the response is fine the way it is, do explain your reasons.

Task 2.4

Repeat the inferencing in Task 2.2, except without context information, give your observations and what you can infer from it.

Repeat the inferencing in Task 2.2 with the model context window set to 100. Give your observations and what you can infer from this.

Bonus task

There might be irrelevant questions asked when referencing the context. Try a way to avoid responding to irrelevant questions. Show the results and describe your implementation.

Assignment 3:

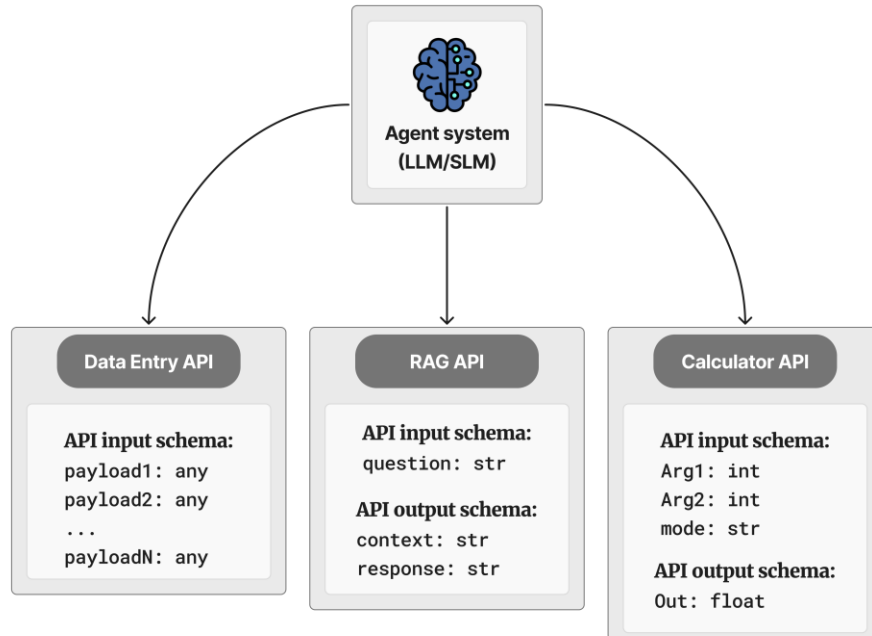
You are trying to run a feasibility study on using agentic systems to serve end users. Your goal is to verify the capability of language model in tool calling for task automation.

Here are the tasks to realize the objective:

1. First, use your favorite Python API framework (FastAPI, Flask, Django etc.) to create a locally hosted API server with a minimum of 3 API endpoints (any more is not mandatory). These API endpoints should perform processing or information fetching. You could always use the ideas below, so you don't have to brainstorm your own idea for API endpoints:
 - a. Simple addition, subtraction, multiplication and division calculator API.
 - b. Simple data entry API: AI agent receives end-user's message and extract keywords which will be used by the API for data entry into a database. Database is not needed. Just print out the received data at your backend. For standardization, the data at the backend must be in JSON format. Example: "John Smith is our latest technician, and he will be stationed at Block A". Results at backend: {"Name": "John Smith", "Position": "technician", "Location": "Block A"}
 - c. Simple RAG services. Use the RAG system from Task 2.2 and host it as an API service to answer questions on renewable energy affairs. You could host only the context retrieval part and perform the inference separately if you want. Freedom is yours.
2. Build your agent system to tie everything together. You could resort to frameworks like LangGraph or off-the-shelf options like CrewAI etc. or design it yourself.
3. Test your system by interacting with your AI agent. Assess if it could call the necessary tools and complete the desired task.

4. State, if any, the pros and cons of your developed agent system.

Overview of Assignment 3:



Submission guidelines

Place all your answers (description of method, justification or terminal output) along with the documented source code (image of source code preferred, labelled according to task) under a single Word or PDF file in the order of assignment and task number.